

SMARTPARK

Resumo executivo — Análise e decisão arquitetural

Por que manter a arquitetura em camadas agora — e quando evoluir para cliente-servidor

O que o SmartPark precisa resolver no estágio atual



Domínio

Controle físico e operacional de estacionamento: reserva, entrada, saída, sensores, cobrança e notificação.



Arquitetura atual

Um único processo Java, em camadas, sem rede e sem banco de dados. Regras concentradas no SmartParkService.



Integrações

Sensor, pagamento, cancela e notificação são simulados como legados externos, acessados por interfaces/adapters.

O maior desafio não é a falta de padrões, e sim garantir que eles produzam o comportamento esperado — veja a seguir.

Padrões declarados nem sempre produziam o comportamento esperado — e os requisitos mostram onde isso mais pesa

Problemas encontrados nos padrões

- **Observer** — Um segundo `subscribe()` substituíu o primeiro, comprometendo a publicação.
- **Strategy** — A interface existia, mas o preço dependia de `if/else` fixo por tipo de veículo.
- **Adapter** — O adapter do sensor herdava da classe legada em vez de usar composição.
- **Abstract Factory** — Retornava `Object` e não era efetivamente instanciada no fluxo.

Requisitos que pressionam a arquitetura

- RF01** Reservar vaga compatível e livre
 - RF02** Entrada só com vaga/sensor sem conflito
 - RF03** Saída com cobrança real e pagamento aprovado
 - RNF01–04** Confiabilidade, consistência, cobrança, extensibilidade
- Exemplo crítico:** se o pagamento for recusado, a saída permanece bloqueada e o estado registra `PAYMENT_ERROR`.

O problema não era só “qual arquitetura escolher”, mas garantir que padrões e regras entregassem confiabilidade, consistência e extensibilidade.

Como a arquitetura atual funciona

ARQUITETURA

As camadas já isolam boa parte das responsabilidades



Patterns

Observer • Strategy • Adapter • Factory • Abstract Factory • Facade

Corrigidos para que o comportamento corresponda às responsabilidades de cada papel.

Legados externos

Sensor • Pagamento • Cancela • Notificação

Acessados por portas/interfaces e adapters, sem acoplamento direto ao serviço.

Matriz de decisão arquitetural

MATRIZ

Base: matriz do DOCX entregue pelo grupo

Critério	Peso	A — Camadas	B — Cliente-servidor	C — Serviços
Desempenho	3	3 / 9	4 / 12	3 / 9
Segurança	3	2 / 6	4 / 12	4 / 12
Manutenibilidade	2	3 / 6	4 / 8	3 / 6
Disponibilidade	2	2 / 4	3 / 6	4 / 8
Complexidade operacional	2	5 / 10	4 / 8	2 / 4

35

A — Camadas

46

B — Cliente-servidor

39

C — Serviços

Por que manter a arquitetura em camadas

DECISÃO

A matriz orienta a análise; a decisão final também considera contexto, custo e risco

Por que essas notas

Desempenho: sem rede, camadas evita latência; B ganharia com API centralizada.

Manutenibilidade: responsabilidades já separadas por patterns; B separa API da regra.

Complexidade operacional: A é a mais simples — projeto pequeno e prazo curto.

Segurança: hoje não há login/controle de acesso; B/C favorecem autenticação.

Disponibilidade: C isola falhas por serviço; A/B compartilham processo/deploy.

DECISÃO ATUAL: MANTER ARQUITETURA EM CAMADAS

Motivo 1 — Estágio atual

Não há necessidade real de múltiplos clientes externos ou carga de produção.

Motivo 2 — Custo e risco

Grupo pequeno, prazo curto: reescrever partes que funcionam aumenta risco sem ganho.

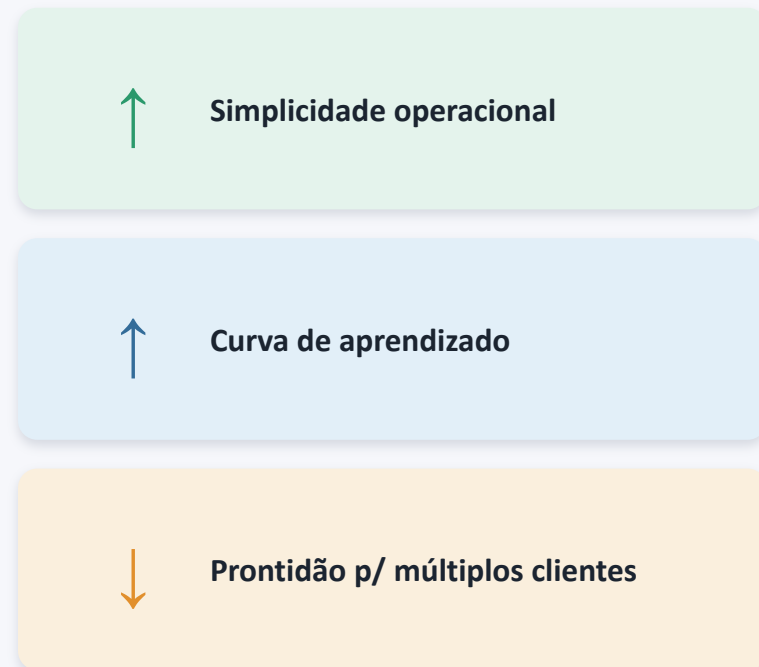
Motivo 3 — Evolução preparada

Regras já isoladas pelos patterns, reduzindo o custo de uma futura migração.

Trade-off aceito e próximo passo

EVOLUÇÃO

A migração para cliente-servidor fica registrada como evolução, não obrigação imediata



Roadmap: cliente-servidor quando houver necessidade

Hoje

Camadas + processo único. Baixa complexidade. Foco em corrigir regras e qualidade.



Gatilho

Necessidade de múltiplos clientes: totem + app + painel. Maior exigência de acesso e escala.



Evolução

API REST, autenticação e separação clara entre interface e negócio.

Por que a migração é viável? As regras de negócio já estão isoladas em padrões e interfaces — a mudança pode se concentrar na exposição/infraestrutura, sem reescrever o núcleo.

Conclusão

1

Corrigir antes de distribuir

Os maiores problemas estavam no comportamento dos padrões e nas regras de negócio: pagamento, estado da vaga, cobrança e integração.

2

Escolher pelo contexto

Cliente-servidor obteve 46 pontos na matriz, mas a decisão considera também custo, prazo, tamanho do projeto e complexidade operacional.

3

Evoluir sem jogar fora

A arquitetura em camadas permanece agora; a migração para cliente-servidor é o próximo passo quando surgir necessidade concreta.

“A melhor arquitetura não é a mais complexa — é a adequada ao problema e ao momento.”